Does DSP Preprocessing Improve AI-Based Environmental Sound Classification?

DSP501 — Digital Signal Processing Final Project
March 2026
Dataset: UrbanSound8K (Salamon et al., 2014)

***Abstract*—This project investigates whether traditional DSP preprocessing improves AI-based environmental sound classification. We design and compare two pipelines on the UrbanSound8K dataset (8,732 clips, 10 classes): Pipeline A feeds raw audio directly to classifiers, while Pipeline B applies FIR bandpass filtering, pre-emphasis, and amplitude normalization before classification. Three models are evaluated—SVM, Random Forest, and CNN-2D—using 10-fold cross-validation with predefined folds. Results show no statistically significant difference between the two pipelines (all $p > 0.05$, all $|d| < 0.2$), suggesting that modern feature extraction methods (MFCCs, mel spectrograms) already capture the relevant information, making explicit DSP preprocessing redundant for this task.**

***Index Terms*—Environmental Sound Classification, Digital Signal Processing, FIR Filter, MFCC, Mel Spectrogram, SVM, Random Forest, CNN, UrbanSound8K**

I. Introduction

A. Motivation

Environmental Sound Classification (ESC) is a fundamental problem in audio signal processing with applications in smart cities, surveillance, wildlife monitoring, and assistive technologies [1]. A key question in the field is whether traditional DSP preprocessing—filtering, pre-emphasis, normalization—provides meaningful improvements when combined with modern machine learning methods.

B. Research Question

> *Does applying classical DSP preprocessing (bandpass filtering, pre-emphasis, normalization) to audio signals improve the accuracy of environmental sound classification compared to using raw audio directly?*

C. Approach

We design two experimental pipelines:

- **Pipeline A** (Baseline): Raw audio → Feature Extraction → Classification
- **Pipeline B** (DSP-Enhanced): Raw audio → FIR Bandpass → Pre-emphasis → Normalization → Feature Extraction → Classification

Both pipelines are evaluated with identical models and evaluation methodology, differing only in whether DSP preprocessing is applied. Three classifiers are tested: SVM, Random Forest, and CNN-2D.

II. Dataset

A. UrbanSound8K

The UrbanSound8K dataset [1] contains 8,732 labeled sound excerpts ($\leq$ 4 seconds) of urban sounds across 10 classes. Table I summarizes the class distribution.

TABLE I: UrbanSound8K class distribution and signal characteristics.

Class	Samples	Category
air_conditioner	1,000	Stationary
car_horn	429	Non-stationary
children_playing	1,000	Stationary-like
dog_bark	1,000	Non-stationary
drilling	1,000	Non-stationary
engine_idling	1,000	Stationary
gun_shot	374	Non-stationary
jackhammer	1,000	Stationary-like
siren	929	Stationary-like
street_music	1,000	Stationary-like
Total	**8,732**	

Key properties:

- 10 predefined folds for cross-validation (never shuffled)
- Class imbalance: *car_horn* (429) and *gun_shot* (374) are underrepresented
- All audio resampled to **22,050 Hz**, padded/truncated to **4 seconds** (88,200 samples)

B. Signal Characteristics

Stationarity analysis reveals two distinct signal groups:

- **Stationary-like**: relatively stable spectral content (air_conditioner, engine_idling, children_playing, jackhammer, siren, street_music)
- **Non-stationary**: impulsive/transient characteristics (car_horn, dog_bark, gun_shot, drilling)

Spectral analysis further groups classes by dominant frequency bands:

- **Low frequency** ($<$ 250 Hz): engine_idling (64.1%), air_conditioner (55.6%)
- **Mid frequency** (250–1,000 Hz): dog_bark (60.5%), car_horn (45.8%)
- **High frequency** (1–4 kHz): children_playing (41.4%), siren (40.9%)
- **Broadband**: drilling, jackhammer, gun_shot

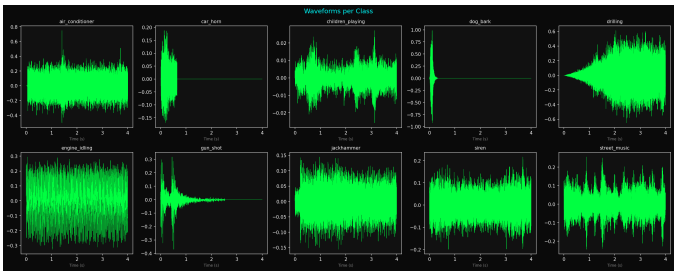

Fig. 1: Raw waveforms for all 10 classes in UrbanSound8K.

Fig. 2: Power spectral density (PSD) per class, showing distinct frequency profiles.

III. DSP Pipeline Design

Pipeline B applies three sequential DSP stages before feature extraction, as illustrated in Figure 3.

$$x[n] \xrightarrow{\text{FIR Bandpass}} x_f[n] \xrightarrow{\text{Pre-emphasis}} x_p[n] \xrightarrow{\text{Normalize}} x_{\text{out}}[n]$$

Fig. 3: Pipeline B processing chain.

A. FIR Bandpass Filter

The FIR bandpass filter is designed using the **window method** with a Hann window. The ideal bandpass impulse response is:

$$h_d[n] = \frac{\sin(\omega_h n)}{\pi n} - \frac{\sin(\omega_l n)}{\pi n} \tag{1}$$

where $\omega_l = 2\pi \cdot 50/22050$ and $\omega_h = 2\pi \cdot 10000/22050$.

Parameters:

- Passband: 50 Hz – 10,000 Hz
- Order: 101 taps
- Window: Hann
- Implementation: zero-phase filtering via `scipy.signal.filtfilt`

Key properties:

- *Linear phase*: $\phi(\omega) = -\frac{M-1}{2}\omega$, preserving temporal structure
- *Constant group delay*: $\tau_g = 50$ samples
- *Always stable*: FIR systems have all poles at origin

The transfer function is:

$$H(z) = \sum_{k=0}^{100} h[k]\, z^{-k} \tag{2}$$

Zero-phase implementation applies forward-backward filtering:

$$y[n] = \mathcal{F}^{-1}\left\{|H(e^{j\omega})|^2 \cdot X(e^{j\omega})\right\} \tag{3}$$

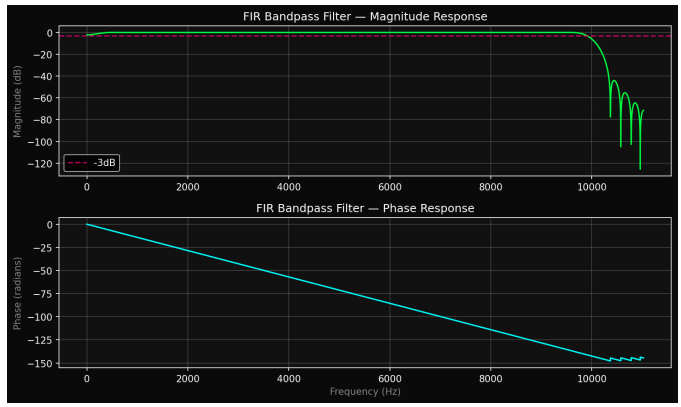

Fig. 4: FIR bandpass filter frequency response: magnitude and phase.

B. IIR Butterworth Filter (Comparison)

For comparison, a 5th-order IIR Butterworth bandpass filter was implemented:

$$|H_a(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2N}} \tag{4}$$

Converted to the digital domain via bilinear transform with frequency pre-warping. The IIR filter is **stable** (all poles inside the unit circle, verified in Figure 6) but has **nonlinear phase**, which distorts temporal structure.

Decision: FIR chosen as the primary filter because linear phase is critical for preserving temporal characteristics of environmental sounds.

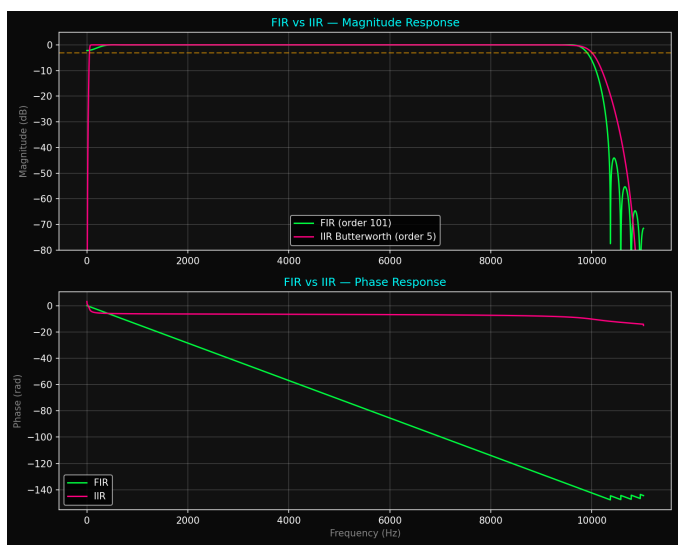

Fig. 5: FIR vs. IIR frequency response comparison.

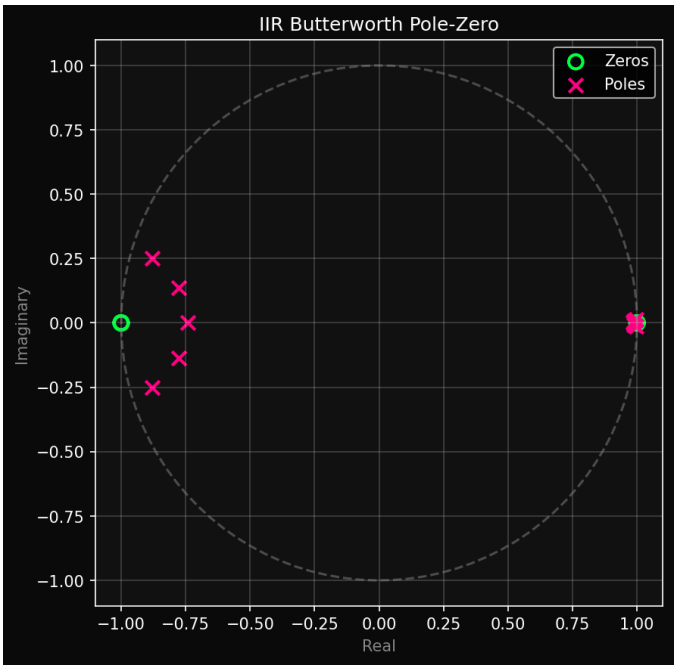

Fig. 6: IIR Butterworth pole-zero diagram. All poles lie inside the unit circle, confirming BIBO stability.

C. Pre-emphasis Filter

A first-order high-pass filter boosts high frequencies:

$$x_p[n] = x[n] - \alpha \cdot x[n-1], \quad \alpha = 0.97 \tag{5}$$

with transfer function $H_{\text{pre}}(z) = 1 - 0.97z^{-1}$.

D. Peak Normalization

Amplitude is scaled to the $[-1, 1]$ range:

$$x_{\text{out}}[n] = \frac{x_p[n]}{\max_n |x_p[n]|} \tag{6}$$

E. DSP Impact on Signal Quality

SNR measurements show DSP preprocessing does improve raw signal quality:

- *children_playing*: $+4.5$ dB
- *jackhammer*: $+2.9$ dB
- Impulsive signals (*car_horn*, *gun_shot*): SNR estimation unreliable due to no quiet frames

IV. Feature Engineering

A. Handcrafted Features (SVM & Random Forest)

A 931-dimensional feature vector is extracted from each audio clip, as detailed in Table II. Seven summary statistics (mean, std, min, max, median, skewness, kurtosis) are computed for each per-frame feature over time.

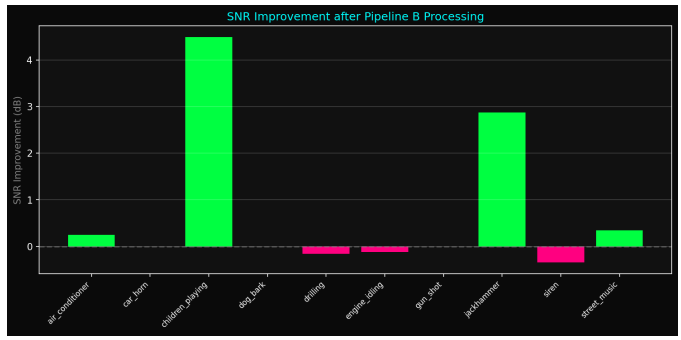

Fig. 7: SNR improvement from DSP preprocessing per class.

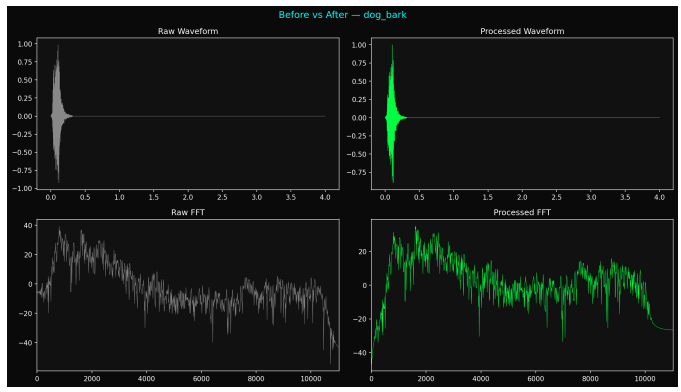

Fig. 8: Example: before and after DSP preprocessing for *dog_bark* class (waveform and spectrogram).

B. Mel Spectrogram (CNN-2D)

For the CNN-2D model, mel spectrograms are extracted with the following parameters:

- 128 mel bands, $f_{\min} = 50$ Hz, $f_{\max} = 10{,}000$ Hz
- FFT size: 2048, hop length: 512
- Output shape: (128×173) per clip
- Converted to log scale (dB), normalized per sample

C. Dimensionality Reduction

For SVM, PCA reduces the feature space from 931 to 200 dimensions. This is necessary because SVM with RBF kernel has computational complexity $O(n^2 d)$, making 931 features prohibitively expensive for ∼7,800 training samples per fold.

TABLE II: Handcrafted feature vector composition (931 dimensions).

Feature Group	Per-Frame	Stats	Dims
MFCC (40) + Δ + ΔΔ	120	7	840
Spectral centroid	1	7	7
Spectral bandwidth	1	7	7
Spectral rolloff	1	7	7
Spectral flatness	1	7	7
Zero-crossing rate	1	7	7
RMS energy	1	7	7
Spectral contrast (7)	7	7	49
Total			**931**

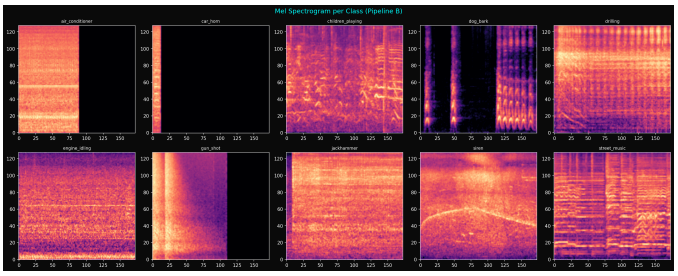

Fig. 9: Mel spectrogram examples for all 10 classes.

Random Forest uses the full 931-dimensional vector, as tree-based methods handle high-dimensional data efficiently.

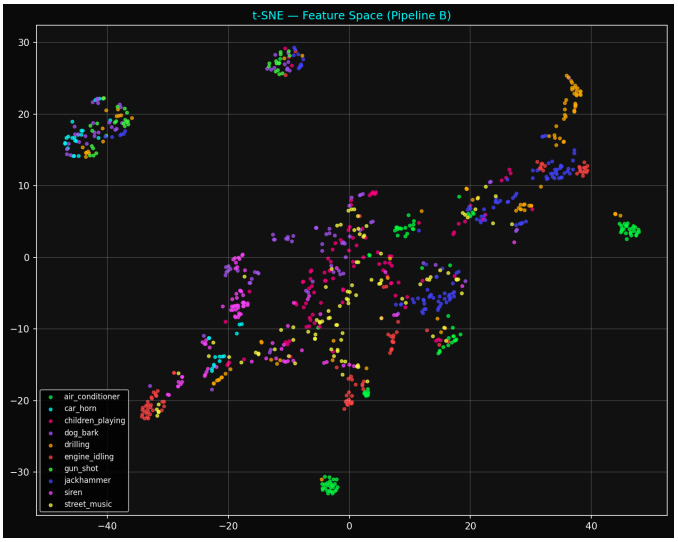

Fig. 10: t-SNE visualization of the 931-dimensional feature space, showing class clustering patterns.

V. Classification Models

A. SVM (Support Vector Machine)

- Kernel: RBF (Radial Basis Function)
- Preprocessing: StandardScaler $\rightarrow$ PCA(200)
- Hyperparameters: $C = 10$, $\gamma = \text{scale}$
- Decision function:

$$f(\mathbf{x}) = \text{sign}\left(\sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right) \tag{7}$$

B. Random Forest

- 500 trees, unlimited depth
- Preprocessing: StandardScaler (no PCA)
- Full 931-dimensional feature vector
- Parallelized across all CPU cores

C. CNN-2D (Convolutional Neural Network)

The CNN architecture processes mel spectrograms of shape $(1 \times 128 \times 173)$ through four convolutional blocks followed by fully connected layers:

Training configuration:

TABLE III: CNN-2D architecture.

Layer	Output Shape
Input	$(1, 128, 173)$
Conv2d$(1 \rightarrow 32, 3 \times 3)$ + BN + ReLU + MaxPool(2)	$(32, 63, 86)$
Conv2d$(32 \rightarrow 64, 3 \times 3)$ + BN + ReLU + MaxPool(2)	$(64, 30, 42)$
Conv2d$(64 \rightarrow 128, 3 \times 3)$ + BN + ReLU + MaxPool(2)	$(128, 14, 20)$
Conv2d$(128 \rightarrow 256, 3 \times 3)$ + BN + ReLU + AdaptiveAvgPool(1)	$(256, 1, 1)$
Flatten + Dropout(0.5)	(256)
Dense$(256 \rightarrow 128)$ + ReLU + Dropout(0.3)	(128)
Dense$(128 \rightarrow 10)$	(10)

- Optimizer: Adam (lr $= 0.001$)
- Loss: CrossEntropyLoss
- Scheduler: ReduceLROnPlateau (patience $= 3$, factor $= 0.5$)
- Early stopping: patience $= 5$ epochs
- Batch size: 64
- Device: Apple MPS (Metal Performance Shaders)

VI. Experimental Methodology

A. Evaluation Protocol

All experiments follow a strict 10-fold cross-validation scheme using UrbanSound8K's predefined folds. Each fold serves as the test set once, with the remaining 9 folds for training. Folds are **never shuffled**—the same samples always remain in the same fold, as per dataset guidelines [1].

B. Metrics

- **Accuracy**: Fraction of correctly classified samples
- **F1 Score (macro)**: Harmonic mean of precision and recall, averaged across classes (handles class imbalance)
- **95% Confidence Interval**:

$$\text{CI} = 1.96 \cdot \frac{\sigma}{\sqrt{n}}, \quad n = 10 \text{ folds} \tag{8}$$

C. Statistical Tests

To determine whether the difference between **Pipeline A** and **Pipeline B** is statistically significant, three tests are applied:

1. Paired t-test: Tests if the mean per-fold difference is significantly different from zero.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad \bar{d} = \frac{1}{n}\sum_{i=1}^{n}(a_i - b_i) \tag{9}$$

2. Wilcoxon signed-rank test: Non-parametric alternative that does not assume normal distribution of differences.

3. Cohen's d (effect size): Quantifies the magnitude of the difference.

$$d = \frac{\bar{d}}{s_d} \tag{10}$$

Interpretation: $|d| < 0.2$ = negligible, 0.2–0.5 = small, 0.5–0.8 = medium, > 0.8 = large.

D. Feature Caching

To avoid repeated feature extraction (∼14 hours), all features were pre-extracted and cached: raw features (8732×931), DSP features (8732×931), raw mel spectrograms ($8732 \times 128 \times 173$), DSP mel spectrograms ($8732 \times 128 \times 173$), labels, and fold assignments.

VII. Results

A. Classification Accuracy

Table IV presents the main accuracy results. Random Forest achieves the best overall performance (71.5%), while CNN-2D shows the lowest accuracy (66.7%) and highest variance.

TABLE IV: Classification accuracy (%) across 10 folds. Best model in bold.

Model	Pipe. A	Pipe. B	Δ	p-value
SVM	70.1 ± 3.2	70.0 ± 3.6	-0.12	0.846
Random Forest	$\mathbf{71.5 \pm 2.6}$	71.2 ± 2.2	-0.26	0.768
CNN-2D	66.7 ± 5.2	67.6 ± 4.8	$+0.94$	0.625

B. F1 Score (Macro)

Table V shows F1 scores, which account for class imbalance. The pattern mirrors accuracy results—no meaningful difference between pipelines.

TABLE V: Macro F1 Score (%) across 10 folds.

Model	Pipe. A	Pipe. B	Δ	p-value
SVM	71.2 ± 3.1	71.2 ± 3.5	-0.04	0.955
Random Forest	72.8 ± 2.2	72.6 ± 1.9	-0.21	0.795
CNN-2D	66.7 ± 5.3	66.8 ± 5.3	$+0.08$	0.966

C. Statistical Significance

Table VI summarizes the full statistical analysis. **None of the three models show a statistically significant difference** at the $\alpha = 0.05$ significance level.

TABLE VI: Statistical significance tests for Pipeline A vs. Pipeline B.

Model	t-test p	Wilcoxon p	Cohen's d	Effect
SVM	0.846	0.557	0.063	Negligible
Random Forest	0.768	0.846	0.096	Negligible
CNN-2D	0.625	0.695	-0.160	Negligible

D. Per-Fold Analysis

Per-fold accuracy shows high variance, particularly for CNN-2D (Figure 12):

- SVM range: 63.4%–78.6% (Pipe. A), 62.8%–78.6% (Pipe. B)
- RF range: 62.7%–78.5% (Pipe. A), 65.9%–78.0% (Pipe. B)
- CNN range: 53.6%–78.0% (Pipe. A), 54.6%–81.1% (Pipe. B)

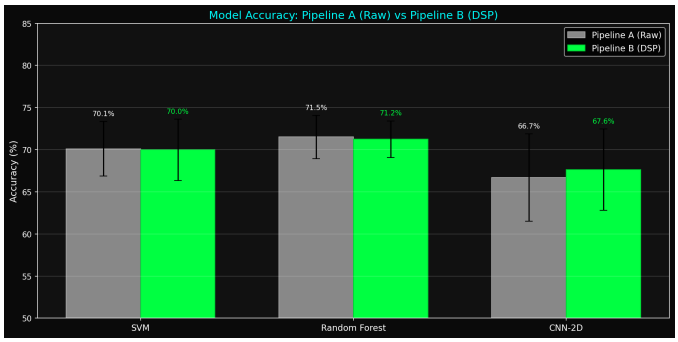

Fig. 11: Pipeline A vs. Pipeline B accuracy comparison across models.

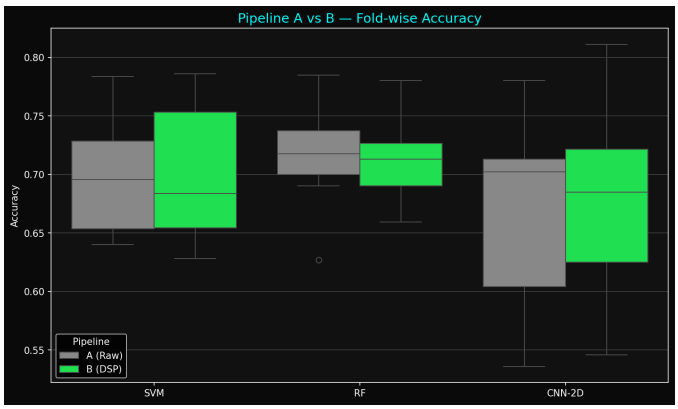

Fig. 12: Box plot of per-fold accuracy distribution for all models and pipelines.

VIII. Analysis and Discussion

A. Why Doesn't DSP Help?

The central finding—that explicit DSP preprocessing provides no significant improvement—can be explained by the **redundancy between DSP preprocessing and modern feature extraction**:

1) **MFCC computation** applies mel-scale filterbanks (effectively bandpass filtering) and computes cepstral coefficients, already emphasizing perceptually relevant frequency bands.
2) **Mel spectrogram parameters** use $f_{\min} = 50$ Hz and $f_{\max} = 10{,}000$ Hz—the *same passband* as our FIR filter. The mel spectrogram inherently ignores frequencies outside this range.

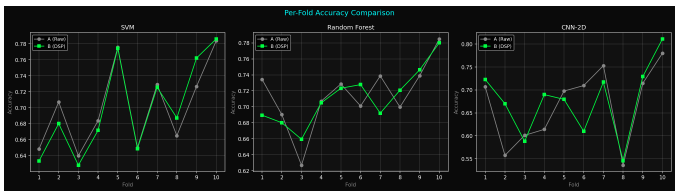

Fig. 13: Per-fold accuracy line plots for all models.

3) **Statistical aggregation** (mean, std, skewness, kurtosis over time frames) provides robustness against noise, similar to what filtering aims to achieve.
4) **StandardScaler** in the ML pipeline normalizes features to zero mean and unit variance, overlapping with the effect of amplitude normalization.

In essence, the feature extraction stage already captures what DSP preprocessing aims to provide, making the explicit DSP step redundant.

B. Model Comparison

- **Random Forest** achieves the best results (71.5%), benefiting from its ability to handle 931 dimensions without PCA and providing interpretable feature importance (Figure 14).
- **SVM** achieves comparable results (70.1%) but required PCA reduction to 200 dimensions for computational feasibility.
- **CNN-2D** underperforms classical ML (66.7%), likely due to the limited dataset size (∼7,800 training samples per fold).

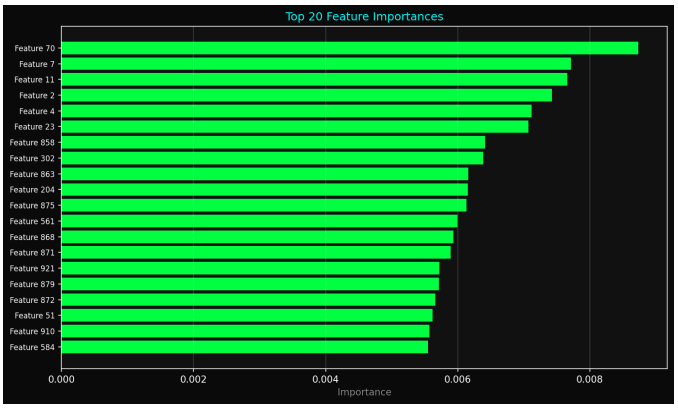

Fig. 14: Random Forest feature importance, showing which features contribute most to classification.

C. CNN Performance Analysis

The CNN-2D exhibits the highest fold-to-fold variance (CI $= 5.2\%$) and lower absolute accuracy than classical ML. This is expected for deep learning on small datasets:

- ∼7,800 training samples is relatively small for a 4-layer CNN
- Early stopping (patience $= 5$) may prevent full convergence on some folds
- Data augmentation (not implemented) could significantly improve CNN performance

D. Limitations

1) **No data augmentation**: CNN performance could improve with time-shifting, pitch-shifting, and noise injection
2) **Fixed SVM hyperparameters**: Exhaustive grid search was limited for computational reasons; optimal C/γ could improve SVM results
3) **No per-class analysis**: Some classes may benefit from DSP more than others
4) **Single filter configuration**: Only one FIR design (50–10,000 Hz, order 101) was tested; different passbands might yield different results
5) **No pre-trained models**: Transfer learning from large audio models (e.g., VGGish, PANNs) was not explored

IX. Conclusion

A. Answer to Research Question

DSP preprocessing does not significantly improve environmental sound classification accuracy. Across all three models (SVM, Random Forest, CNN-2D), the differences between **Pipeline A** (raw) and **Pipeline B** (DSP-preprocessed) are:

- Not statistically significant (all $p > 0.05$)
- Negligible in effect size (all $|d| < 0.2$)
- Practically insignificant ($|\Delta| < 1\%$)

B. Key Takeaways

1) Modern feature extraction (MFCCs, mel spectrograms) already implicitly performs the filtering and normalization that explicit DSP preprocessing provides.
2) Classical ML (Random Forest: 71.5%) outperforms deep learning (CNN-2D: 66.7%) on this dataset size (∼8,700 samples).
3) The FIR bandpass filter does improve raw signal SNR ($+2.9$ to $+4.5$ dB for some classes), but this improvement does not translate to better classification accuracy.
4) DSP preprocessing is not wasted—understanding filter design, frequency analysis, and signal characteristics provides valuable engineering insight even when the classification improvement is negligible.

C. Future Work

- Apply data augmentation for CNN training (time-shift, pitch-shift, noise injection)
- Test class-specific DSP preprocessing (e.g., different filters for stationary vs. non-stationary sounds)
- Experiment with deeper architectures or pre-trained models (VGGish, PANNs)
- Investigate attention mechanisms for variable-length audio
- Evaluate on larger datasets (ESC-50, AudioSet) to test generalizability

Acknowledgments

This project uses the UrbanSound8K dataset provided by Salamon et al. [1] via the Urban Sound Taxonomy project at New York University.

References

[1] J. Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 1041–1044.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.

[3] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.

[4] K. J. Piczak, "Environmental sound classification with convolutional neural networks," in *Proc. IEEE 25th Int. Workshop Mach. Learn. Signal Process. (MLSP)*, 2015, pp. 1–6.

Appendix

A. Environment

- Python 3.13.2
- PyTorch 2.6 (MPS backend for macOS)
- scikit-learn 1.6
- librosa 0.10
- scipy 1.15

B. Random Seeds

All experiments use `RANDOM_SEED = 42`. NumPy, PyTorch, and scikit-learn seeds are set before each fold for full reproducibility.

C. Data Processing

UrbanSound8K v2 from Zenodo. 10 predefined folds, never shuffled. Audio resampled to 22,050 Hz, padded/truncated to 4 seconds (88,200 samples).

D. Code

All source code is available in the `src/` directory. Jupyter notebooks in `notebooks/` reproduce the full pipeline from raw data to final results. Run notebooks sequentially: `01` → `02` → `03` → `04` → `05` → `06`.

E. All Hyperparameters

TABLE VII: Complete hyperparameter configuration.

Parameter	**Value**
Audio	
Sample rate	22,050 Hz
Duration	4 s (88,200 samples)
DSP	
FIR passband	50–10,000 Hz
FIR order	101 taps
IIR order	5 (Butterworth)
Pre-emphasis α	0.97
Features	
MFCC coefficients	40
Mel bands	128
FFT size	2048
Hop length	512
SVM	
Kernel	RBF
C	10
γ	scale
PCA components	200
Random Forest	
Trees	500
Max depth	None (unlimited)
CNN-2D	
Epochs	100 (early stop)
Batch size	64
Learning rate	0.001
Patience	5